

Aetheris AI — Flow & UI Change Specification

1. Objective

Modify the existing Aetheris AI application flow to support **role-specific onboarding, integrations, sidebar content, and dashboard behavior**.

The application currently supports multiple user roles. After this change, the experience should differ based on the selected role.

Roles

1. Developer
2. Content Creator
3. Freelancer
4. Start-Up

The existing functionality and dashboard for **Developer** and **Start-Up** must remain unchanged.

The new functionality will primarily apply to:

- Content Creator → Instagram integration + DM assistant
- Freelancer → Gmail integration + Email assistant

2. Important Implementation Rules

Before making changes:

- **Do not rewrite the existing application architecture unnecessarily.**
 - Reuse existing components, layouts, authentication, database structure, API utilities, and styling wherever possible.
 - Do not break existing Developer or Start-Up functionality.
 - Do not remove existing features unless explicitly required in this document.
 - Keep the UI visually consistent with the existing Aetheris AI design system.
 - Make the new functionality modular so that additional integrations can be added later.
 - Role-specific functionality must be determined from the authenticated user's selected role.
 - Do not hard-code a role for testing in production code.
 - Existing users should continue to work without being forced through a broken onboarding flow.
-

3. New Onboarding Flow

3.1 Role Selection

The onboarding page must display four role options:

- Developer
- Content Creator
- Freelancer
- Start-Up

The role selection should be stored against the authenticated user so the application can determine which dashboard and integrations to display.

3.2 Developer Role

When the user selects **Developer**:

1. Display a popup/modal message:
"This feature is currently in development."
 1. Do not redirect the user to a new Content Creator/Freelancer onboarding flow.
 2. Keep the existing Developer dashboard unchanged.
 3. Keep the existing Developer sidebar unchanged.
 4. Keep existing Developer functionality unchanged.
-

3.3 Start-Up Role

When the user selects **Start-Up**:

1. Display a popup/modal message:
"This feature is currently in development."
1. Do not redirect the user to a new Content Creator/Freelancer onboarding flow.
2. Keep the existing Start-Up dashboard unchanged.
3. Keep the existing Start-Up sidebar unchanged.

4. Keep existing Start-Up functionality unchanged.
-

4. Content Creator Flow

4.1 Onboarding

When the user selects **Content Creator**:

1. Save the selected role.
2. Redirect the user to an **Instagram Integration** screen.
3. Explain that Instagram access is required to retrieve and manage Instagram Direct Messages.
4. Provide an Instagram connection/authentication button.
5. After successful authentication, store the required integration information securely.
6. Redirect the user to the Content Creator dashboard.

Integration Requirements

The integration must:

- Use the official supported authentication/API flow.
- Request only the permissions required by the application.
- Handle authentication failure gracefully.
- Handle expired/revoked access gracefully.
- Never expose access tokens or sensitive credentials in the frontend.
- Avoid storing sensitive credentials directly in client-side storage.

If the Instagram integration cannot currently be completed because the required API permissions/configuration are unavailable, create the integration flow in a modular way and provide an appropriate connection/error state rather than implementing a fake integration.

5. Content Creator Dashboard

After Instagram integration, the dashboard should become an **Instagram DM AI workspace**.

5.1 Dashboard Overview

The dashboard should display:

Inbox Statistics

- Total number of Instagram DMs/messages available.
- Number of messages requiring attention, if this information can be reliably determined.
- Number of AI-generated replies.

Do not invent statistics if the Instagram API does not provide the required data.

5.2 Message Analysis

For each relevant Instagram DM/conversation:

1. Retrieve the available message content.
2. Display the conversation/message in the appropriate UI.
3. Send the message content to the AI processing layer.
4. Generate a suggested reply based on the conversation context.
5. Display the AI-generated reply to the user.

The AI should consider:

- Message content
 - Previous conversation context, when available
 - Sender/conversation context
 - User instructions
 - Existing AI/system instructions
-

6. Content Creator Input Panel

The existing AI input panel should be reused.

The user must be able to provide additional instructions for modifying the AI-generated response.

Example:

"Make the response more professional."

or:

"Make this reply shorter and more friendly."

The flow should be:

Instagram DM → AI analysis → Suggested Reply → User Prompt → Revised Reply

The user should be able to regenerate/edit the response without losing the original conversation context.

7. Freelancer Flow

7.1 Onboarding

When the user selects **Freelancer**:

1. Save the selected role.
2. Redirect the user to a **Gmail Integration** screen.
3. Explain that Gmail access is required to retrieve and analyse emails.
4. Provide a Gmail/Google authentication button.
5. After successful authentication, securely store the required integration information.
6. Redirect the user to the Freelancer dashboard.

Gmail Integration Requirements

The integration must:

- Use Google's official OAuth flow.
- Request only the minimum required Gmail permissions.
- Handle authentication failure gracefully.
- Handle expired/revoked access gracefully.
- Never expose access tokens or secrets in the frontend.
- Never store sensitive credentials directly in client-side storage.

If the required Gmail API configuration is unavailable, implement the integration architecture and UI states without creating fake email data.

8. Freelancer Dashboard

After Gmail integration, the dashboard should become an **AI Email Assistant workspace**.

8.1 Dashboard Overview

Display:

Inbox Statistics

- Total number of relevant emails in the inbox.
- Number of emails requiring attention, if reliably available.
- Number of AI-generated replies.

Avoid displaying fabricated or estimated values.

8.2 Email Summary

For each relevant email:

1. Retrieve the email.
2. Display the sender.
3. Display the subject.
4. Display the received date/time.
5. Display the relevant email content.
6. Generate a concise AI summary.
7. Analyse the intent/context of the email.
8. Generate a suggested reply.

The AI should consider:

- Sender
 - Subject
 - Email body
 - Conversation/thread context when available
 - User instructions
 - Existing AI/system instructions
-

9. Freelancer Input Panel

Reuse the existing AI input panel.

The user should be able to provide instructions for modifying the generated email response.

Examples:

"Make this email more professional."

"Make it shorter."

"Accept the proposal but ask for a higher budget."

"Make the tone friendly but professional."

The flow should be:

Gmail Email → AI Summary/Analysis → Suggested Reply → User Prompt → Revised Reply

The original email and conversation context must remain available while generating the revised response.

10. Sidebar Changes

The sidebar behavior must depend on the selected user role.

10.1 Developer

Keep the existing Developer sidebar exactly as it currently works.

No changes unless required for compatibility.

10.2 Start-Up

Keep the existing Start-Up sidebar exactly as it currently works.

No changes unless required for compatibility.

10.3 Freelancer

The sidebar should display the user's Gmail inbox/email conversations.

Each sidebar item should preferably contain:

- Sender name/email
- Email subject
- Short preview/snippet
- Date/time
- Read/unread state, if available

Selecting an email should load its complete context into the main workspace.

The sidebar should not unnecessarily load the full body of every email at once. Fetch detailed email content when required.

10.4 Content Creator

The sidebar should display the user's Instagram DM conversations.

Each sidebar item should preferably contain:

- Instagram username/name
- Message preview

- Latest message time
- Unread state, if available

Selecting a conversation should load the relevant conversation context into the main workspace.

11. Conversation History

The existing **Clear Conversation History** functionality should remain available.

However, distinguish between:

A. AI Chat/Prompt History

Conversation generated through the Aetheris AI input panel.

B. External Integration Data

Emails and Instagram DMs retrieved from Gmail/Instagram.

Clearing Aetheris AI conversation history must **not automatically delete the user's actual Gmail emails or Instagram messages**.

The application should only remove/reset locally stored AI conversation data unless the user explicitly performs an external action supported by the respective API.

12. Main Workspace Behavior

The main workspace should dynamically adapt to the selected role.

Developer

Existing workspace → unchanged.

Start-Up

Existing workspace → unchanged.

Freelancer

Main workspace:

Email selected from sidebar

→ Email content

- AI summary
- AI analysis
- Suggested reply
- User instruction input
- Revised reply

Content Creator

Main workspace:

Instagram DM selected from sidebar

- Conversation content
 - AI analysis
 - Suggested reply
 - User instruction input
 - Revised reply
-

13. Data & State Requirements

The application should maintain a clear separation between:

User Data

- User ID
- Selected role
- Onboarding status

Integration Data

- Integration type
- Connection status
- Provider account identifier
- Secure authentication information

External Content

- Gmail emails
- Instagram conversations/messages

AI Data

- AI analysis
- Generated summaries
- Generated replies
- User prompts
- Revised replies

Use the existing database architecture where possible.

Do not duplicate data unnecessarily.

14. Loading, Error & Empty States

Every integration-dependent screen must handle:

Loading

Example:

"Syncing your inbox..."

Empty Inbox

Example:

"No new emails found."

Empty Instagram Inbox

Example:

"No messages found."

Authentication Failure

Example:

"We couldn't connect your account. Please try again."

Expired Connection

Example:

"Your account connection has expired. Reconnect to continue."

API Failure

Example:

"We couldn't retrieve your messages right now. Please try again."

AI Generation Failure

Example:

"We couldn't generate a response. Try again or edit the message manually."

Do not leave the user with a blank screen or broken component.

15. Security Requirements

Because Gmail and Instagram integrations involve user data:

- Never expose OAuth access tokens in the UI.
 - Never hard-code API keys/secrets.
 - Do not store secrets in frontend code.
 - Use environment variables for application secrets.
 - Follow the existing authentication/security architecture.
 - Apply appropriate database access controls.
 - Users must only be able to access their own integration data.
 - Do not allow one user's emails/messages to be accessible by another user.
 - Log errors without exposing sensitive email/message content or authentication credentials.
-

16. UI/UX Requirements

The new Content Creator and Freelancer experiences should feel like extensions of the existing Aetheris AI interface.

Do not create an unrelated UI design.

Maintain:

- Existing typography
- Existing color system
- Existing spacing system
- Existing components
- Existing sidebar behavior where applicable
- Existing AI input/output experience
- Existing responsive behavior

Only introduce new UI components where required for:

- Integration
- Email/DM lists

- Email/DM details
 - AI summaries
 - AI-generated replies
 - Integration status
 - Inbox statistics
-

17. Backward Compatibility

This change must not break:

- Existing authentication
- Existing Developer dashboard
- Existing Start-Up dashboard
- Existing Developer sidebar
- Existing Start-Up sidebar
- Existing AI functionality
- Existing database functionality
- Existing settings/configuration
- Existing routes that are still required

If an existing component is shared between roles, modify it using role-based conditional rendering rather than creating unnecessary duplicate components.

18. Implementation Approach

Before coding:

1. Analyse the existing project structure.
2. Identify the current onboarding flow.
3. Identify how user roles are stored.
4. Identify the current dashboard routing.
5. Identify the existing sidebar architecture.
6. Identify the existing AI input/output components.
7. Identify the current database schema.
8. Identify existing authentication mechanisms.
9. Identify whether Gmail/Instagram integration infrastructure already exists.
10. Identify reusable components before creating new ones.

Then implement the changes incrementally.

Recommended implementation order

Phase 1 — Existing Flow Analysis

- Understand current architecture
- Identify affected files/components
- Do not modify code yet

Phase 2 — Role-Based Onboarding

- Add/modify four role selection options
- Add Developer/Start-Up development modal
- Preserve existing dashboards

Phase 3 — Content Creator

- Instagram integration screen
- Instagram authentication
- DM retrieval layer
- DM sidebar
- Message workspace
- AI analysis
- AI reply generation
- Prompt-based reply refinement

Phase 4 — Freelancer

- Gmail integration screen
- Gmail authentication
- Email retrieval layer
- Email sidebar
- Email workspace
- AI summarisation
- AI analysis
- AI reply generation
- Prompt-based reply refinement

Phase 5 — Dashboard & Statistics

- Inbox/message counts
- AI reply counts
- Loading/empty/error states

Phase 6 — Security & Data Isolation

- Verify authentication
- Verify database access policies
- Verify user-specific data isolation
- Verify OAuth token handling

Phase 7 — Testing

Test each role independently:

Developer

- Role selection
- Development popup
- Existing dashboard
- Existing sidebar
- Existing functionality

Start-Up

- Role selection
- Development popup
- Existing dashboard
- Existing sidebar
- Existing functionality

Content Creator

- Role selection
- Instagram integration
- Successful authentication
- Failed authentication
- DM retrieval
- Sidebar conversation selection
- AI analysis
- Reply generation
- Reply refinement
- Empty inbox
- API failure
- Expired connection

Freelancer

- Role selection
- Gmail integration
- Successful authentication
- Failed authentication
- Email retrieval
- Sidebar email selection
- Email summary
- AI analysis
- Reply generation
- Reply refinement
- Empty inbox

- API failure
 - Expired connection
-

19. Definition of Done

The implementation is considered complete only when:

- [] Four roles are available during onboarding.
 - [] Developer shows the development-phase message and retains the existing flow.
 - [] Start-Up shows the development-phase message and retains the existing flow.
 - [] Content Creator is redirected to Instagram integration.
 - [] Freelancer is redirected to Gmail integration.
 - [] Connected Instagram DMs appear in the Content Creator sidebar.
 - [] Connected Gmail emails appear in the Freelancer sidebar.
 - [] Selecting a DM/email loads its context into the main workspace.
 - [] AI can analyse the selected content.
 - [] AI can generate a suggested response.
 - [] User can provide additional instructions to modify the response.
 - [] Dashboard statistics are based on real available data.
 - [] Clear Conversation History does not delete external Gmail/Instagram data.
 - [] Loading, empty, authentication, API and AI-error states are handled.
 - [] User data is isolated between accounts.
 - [] Developer and Start-Up functionality remains intact.
 - [] Existing UI/design system is preserved.
 - [] No fake API responses or hard-coded production data are introduced.
 - [] Application builds successfully.
 - [] No existing critical functionality is broken.
-

20. Important Instruction to the Agent

Do not start by blindly modifying the code.

First inspect the existing Aetheris AI project and determine:

- Current folder structure
- Current onboarding implementation
- Current role management
- Current routing
- Current dashboard components
- Current sidebar components
- Current AI input/output components
- Current database schema
- Current authentication implementation

- Existing integration/API infrastructure

Then provide a concise implementation plan identifying:

1. Files/components that need modification
2. Files/components that need to be created
3. Database changes required
4. API/integration changes required
5. Potential breaking changes
6. Security considerations
7. Testing requirements

Only after analysing the existing implementation should coding begin.

Priority: Preserve existing functionality first, then introduce the new role-specific workflows in a modular and maintainable way.